



SIMATS Online Programming Lab
JAVA PROGRAMMING

 SANGEPU VENKATA PRAVEEN
192373013RD

QUESTIONS 8 / 8 completed

1-10 T

Write a Java program to create a simple thread that prints the numbers from 1 to 10.

PROGRAM EDITOR

Java

Run

Save

```
1 public class Main {
2     public static void main(String[] args) {
3         MyThread t = new MyThread();
4         t.start();
5     }
6 }
7
8 class MyThread extends Thread {
9     public void run() {
10         for (int i = 1; i <= 10; i++) {
11             System.out.println(i);
12         }
13     }
14 }
```

INPUT

Your INPUT goes here!

```
1
2
3
4
5
6
7
8
9
10
```



SIMATS Online Programming Lab
JAVA PROGRAMMING

 SANGEPU VENKATA PRAVEEN
192373013RD

QUESTIONS 8 / 8 completed

Counter-T

Write a Java program to create a thread-safe counter that can be accessed by multiple threads simultaneously.

PROGRAM EDITOR

Java

Run

Save

```
1 public class Main {
2     public static void main(String[] args) throws InterruptedException {
3         Counter counter = new Counter();
4         Runnable task = () -> {
5             for (int i = 0; i < 1000; i++) {
6                 counter.increment();
7             }
8         };
9         Thread t1 = new Thread(task);
10        Thread t2 = new Thread(task);
11        t1.start();
12        t2.start();
13        t1.join();
14        t2.join();
15        System.out.println("Final Counter Value: " + counter.getCount());
16    }
17 }
18 class Counter {
19     private int count = 0;
20
21     public synchronized void increment() {
```

INPUT

Your INPUT goes here!

OUTPUT

```
Final Counter Value: 2000
```

**SIMATS**
ENGINEERING

SIMATS Online Programming Lab
JAVA PROGRAMMING

 SANGEPU VENKATA PRAVEEN
192373013RD

QUESTIONS 8 / 8 completed

M-Thread

M-THREAD
Write a Java program to create multiple threads that perform some time-consuming task, such as calculating prime numbers or sorting an array, and compare their execution times.

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.ArrayList;
2 import java.util.List;
3 public class Main {
4     public static void main(String[] args) {
5         int limit = 100000;
6         int numThreads = 4;
7         int rangePerThread = limit / numThreads;
8         List<Integer> allPrimes = new ArrayList<>();
9         List<Thread> threads = new ArrayList<>();
10        long startTime = System.currentTimeMillis();
11        for (int i = 0; i < numThreads; i++) {
12            int start = i * rangePerThread + 1;
13            int end = (i == numThreads - 1) ? limit : (start + rangePerThread - 1);
14            Thread thread = new Thread(new PrimeTask(start, end, allPrimes));
15            threads.add(thread);
16            thread.start();
17        }
18        for (Thread thread : threads) {
19            try {
20                thread.join();
21            } catch (InterruptedException e) {}
22        }
23        long endTime = System.currentTimeMillis();
24        long executionTime = endTime - startTime;
25        System.out.println("Total primes found: " + allPrimes.size());
26        System.out.println("Execution time with " + numThreads + " threads: " + executionTime + " ms");
27    }
28 }
```

INPUT

Your INPUT goes here!

OUTPUT

```
Total primes found: 9592
Execution time with 4 threads: 26 ms
```



SIMATS Online Programming Lab
JAVA PROGRAMMING

 SANGEPU VENKATA PRAVEEN
192373013RD

QUESTIONS 8 / 8 completed

Pro-Consume

PRO-CONSUME
Write a Java program to create a producer-consumer scenario using multiple threads. The producer thread should add items to a queue, and the consumer thread should remove items from the queue and process them.

PROGRAM EDITOR

Java

Run Save

```
1 public class Main {
2     private static final java.util.concurrent.BlockingQueue<Integer> queue = new java.
3     private static final int ITEMS_TO_PRODUCE = 10;
4     public static void main(String[] args) {
5         Thread producerThread = new Thread(new Producer());
6         Thread consumerThread = new Thread(new Consumer());
7         producerThread.start();
8         consumerThread.start();
9         try {
10             producerThread.join();
11             consumerThread.join();
12         } catch (InterruptedException e) {
13             Thread.currentThread().interrupt();
14         }
15     }
16     static class Producer implements Runnable {
17         @Override
18         public void run() {
19             try {
20                 for (int i = 1; i <= ITEMS_TO_PRODUCE; i++) {
```

INPUT
Your INPUT goes here!

```
Consumed: 1
Produced: 1
Produced: 2
Consumed: 2
Produced: 3
Produced: 4
Consumed: 3
Produced: 5
Produced: 6
Consumed: 4
```



SIMATS Online Programming Lab
JAVA PROGRAMMING

 SANGEPU VENKATA PRAVEEN
192373013RD

Deadlock

DEADLOCK
Write a Java program to create a deadlock scenario using multiple threads.

PROGRAM EDITOR

Java

Run Save

```
1 public class Main {
2     public static void main(String[] args) {
3         final Object resource1 = new Object();
4         final Object resource2 = new Object();
5         Thread t1 = new Thread(() -> {
6             synchronized (resource1) {
7                 System.out.println("Thread 1: locked Resource 1");
8                 try { Thread.sleep(100); } catch (InterruptedException e) {}
9                 System.out.println("Thread 1: waiting for Resource 2");
10                synchronized (resource2) {
11                    System.out.println("Thread 1: locked Resource 2");
12                }
13            }
14        });
15        Thread t2 = new Thread(() -> {
16            synchronized (resource2) {
17                System.out.println("Thread 2: locked Resource 2");
18                try { Thread.sleep(100); } catch (InterruptedException e) {}
19                System.out.println("Thread 2: waiting for Resource 1");
20                synchronized (resource1) {
21                    System.out.println("Thread 2: locked Resource 1");
```

INPUT
Your INPUT goes here!

QUESTIONS 8 / 8 completed

Wait-Notify ▾

WAIT-NOTIFY

Write a Java program to demonstrate how to use the "wait" and "notify" methods to implement a producer-consumer scenario.

PROGRAM EDITOR

Java ▾

Run

Save

INPUT

Your INPUT goes here!

```
1 public class Main {
2     static class SharedResource {
3         private int data;
4         private boolean hasData = false;
5         public synchronized void produce(int value) {
6             while (hasData) {
7                 try {
8                     wait();
9                 } catch (InterruptedException e) {
10                     Thread.currentThread().interrupt();
11                 }
12             }
13             data = value;
14             hasData = true;
15             System.out.println("Produced: " + data);
16             notify();
17         }
18         public synchronized void consume() {
19             while (!hasData) {
20                 try {
21                     wait();
```

OUTPUT

```
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
```

QUESTIONS 8 / 8 completed

Pool

POOL

Write a Java program to create a thread pool that can execute multiple threads concurrently.

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.concurrent.ExecutorService;
2 import java.util.concurrent.Executors;
3 import java.util.concurrent.TimeUnit;
4 public class Main {
5     public static void main(String[] args) {
6         ExecutorService pool = Executors.newFixedThreadPool(4);
7         for (int i = 1; i <= 8; i++) {
8             final int taskId = i;
9             pool.execute(() -> {
10                 System.out.println("Task " + taskId + " executed by: " +
11                     Thread.currentThread().getName());
12                 try {
13                     Thread.sleep(300);
14                 } catch (InterruptedException e) {
15                     Thread.currentThread().interrupt();
16                 }
17             });
18         }
19         pool.shutdown();
20         try {
21             if (!pool.awaitTermination(5, TimeUnit.SECONDS)) {
```

INPUT

Your INPUT goes here!

OUTPUT

```
Task 2 executed by: pool-1-thread-2
Task 1 executed by: pool-1-thread-1
Task 3 executed by: pool-1-thread-3
Task 4 executed by: pool-1-thread-4
Task 5 executed by: pool-1-thread-3
Task 6 executed by: pool-1-thread-2
Task 8 executed by: pool-1-thread-1
Task 7 executed by: pool-1-thread-4
```

QUESTIONS 8 / 8 completed

Join

JOIN

Write a Java program to demonstrate how to use the "join" method to wait for a thread to complete its execution before proceeding with the main thread.

PROGRAM EDITOR

Java

Run

Save

```
1 public class Main {
2     public static void main(String[] args) {
3         Thread thread = new Thread(() -> {
4             try {
5                 System.out.println("Child thread is performing work...");
6                 Thread.sleep(2000);
7                 System.out.println("Child thread has completed its task.");
8             } catch (InterruptedException e) {
9                 System.out.println("Thread interrupted: " + e);
10            }
11        });
12        thread.start();
13        try {
14            System.out.println("Main thread is waiting for the child thread to complete");
15            thread.join();
16        } catch (InterruptedException e) {
17            System.out.println("Main thread interrupted: " + e);
18        }
19        System.out.println("Main thread is now proceeding.");
20    }
21 }
```

INPUT

Your INPUT goes here!

OUTPUT

```
Main thread is waiting for the child thread to complete...  
Child thread is performing work...  
Child thread has completed its task.  
Main thread is now proceeding.
```